

Advanced Topics in Software Engineering Assessment 3 Brief

Submission type and weight: Individual, and 40%

Due Date: 28 May 2025 by 23:00

Given Date: 30 April 2026 at 00:00

Submission Format: One PDF report (via Turnitin) and one publicly accessible video demonstration link (e.g., YouTube) clearly shared on the report cover page.

Overview

This is your final assessment (A3) for the Advanced Topics in Software Engineering module. You are required to apply a plan-driven software development model (e.g., Waterfall, Spiral, V-Model, DSDM) to design and partially implement a prototype seating algorithm system for a venue-based business (e.g., cinema, opera house, airplane, auditorium).

Your task includes:

- **Producing UML-based design documentation**
- **Creating a functional and non-functional requirements specification**
- **A seating plan and some fundamental requirements are given to you to consider for your algorithm.**
- **Prototyping a system that reflects your design**
- **Recording and submitting a video demonstration; write a report to document the above**

Submission Requirements: Video Demonstration and Report

Both the report and the video demonstration are mandatory and an integral part of this assessment. **If the video is not submitted or is inaccessible, no mark will be awarded.** The report alone does not adequately communicate your process and solution.

AI Usage and Similarity Policy

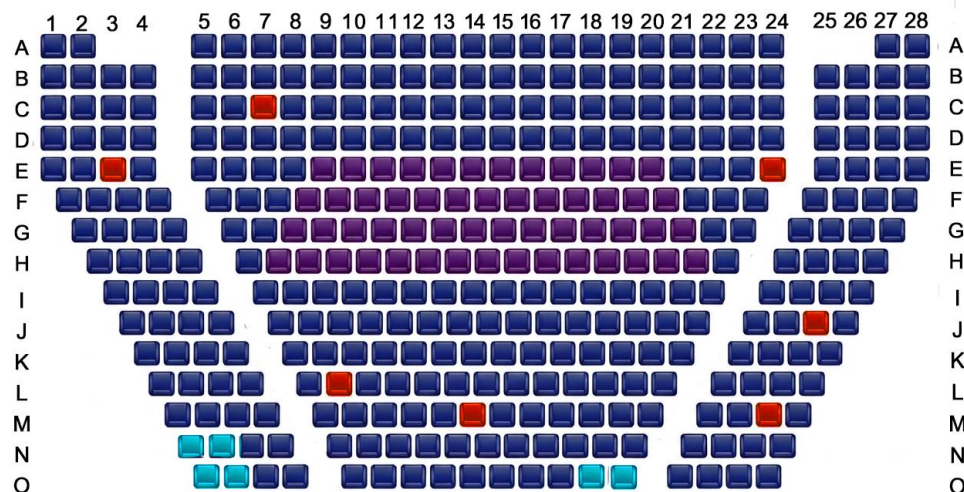
This assessment, unlike the previous two, strictly monitors similarity and AI usage in both the report and video demonstration. **A similarity score of Turnitin above 30% puts your work at risk of being considered plagiarism, and your work might be treated as academic misconduct.** You are highly encouraged to do the work on your own rather than using generative AI, as generative AI can produce solutions, but they often do not produce effective, senior-level solutions to problems due to their inability of working memory to track down multiple constraints such as software architecture, technology, and also the requirements. Therefore, **do not use AI to design the logic of the algorithm, and rather complete the problem solving/thought process yourself with pen and paper if you can**, and once you start writing the code, get minor help from AI. **Overuse of AI in this assessment is prohibited, and you will be capped if this is detected.** You should not use AI to handle all the problem-solving part, write the report, create the video, or for any other purpose in this assessment. As a matter of fact, the problem here is so complicated that AI will misguide you, as in 2016, most AI like

ChatGPT is so merged with Reinforcement Learning from Human Feedback (RLHF) that they act like hall monitors with an incredibly argumentative attitude. Therefore, it is very easy to fall into a pitfall and submit AI-generated content confidently which does not even address the crucial parts of this assessment. Furthermore, this might result in high similarity with others at the code level, which again is a strong indication that generative AI is used, as AI tends to use the same set of variables, data units, same structures, and collapse in them as it is a token-based system. **This assessment prioritizes problem solving, not code usage or AI usage ability.** Completing it successfully requires original thought, critical analysis, and creativity. Ensure that your report is written clearly, in your own words, and accurately reflects your understanding and process. All work must be entirely your own.

Assessment Context

You are tasked with developing an effective algorithm for the usage of a seating plan (select one), where minimising the single scattered seats appearing in the algorithm is your priority in order to optimise the seating plan and use it in the most efficient way. **Please choose one of the seating plans which are given below and start building your solutions.** Please be aware that you are not allowed to design your own seating plan and that you should use one of the three choices below:

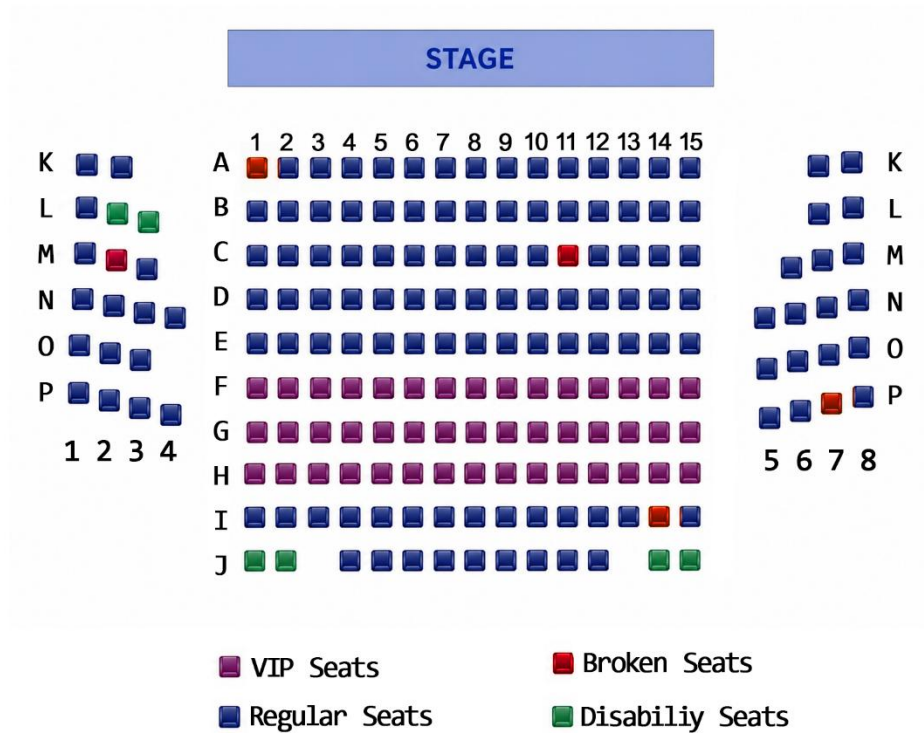
Cinemas (Theatres)



Screen

- vip seat (always in between row E and row I and 4 rows which are 12, 13, 14, and 15 seats)
- regular seat (if its not a defined seat, its a regular seat)
- disability seat (6 random distributed seats but always in the first two front rows and always adjacent)
- broken seat (randomly broken seats in each session in between 6 to 10 but not more than 2 in each row, cannot be adjacent)

Lecture Hall (Auditorium)



Disability Seats: Random 6 disability seats which are always adjacent and can only be in row J, L or K

VIP Seats: Adjacent random 3 rows; from E to H.

Broken Seats: Every session, five seats are randomly marked as broken, but these seats are never VIP or disability seats.

Regular Seats: Every other seat is a regular seat, strictly follow the above seating plan.

Aircraft (Flight Booking)

A	B	C		D	E	F
			1			
			2			
			3			
			4			
			5			
			6			
			7			
			8			
			9			
			10			
			11			
			12			
			13			
			14			
			15			
			16			
			17			
			18			
			19			
			20			
			21			
			22			
			23			
			24			
			25			
			26			
			27			
			28			
			29			
			30			

-  Business Class
-  Economic Class
-  Extra-legroom Seats
-  Mature (No Children) Seats
-  Randomly broken seats

Each Flight has 4 randomly broken seats
However, broken seats are never
among VIP seats or extra leg seats.

When Plane is 70% or more full,
All seats on row 16 and 17 get 50% discount.

A senior passenger can select Economy,
No-Children, or Extra-Legroom seats
regardless of the scattered-seats algorithm
and pays the same price.

Strintly follow the seating plan:

Row 1-5 Business;
Row 6 -15 and 18 -27 Eco
Row 16-17 extra-legroom seats
Row 28 to 30 No Children (<= 18 only)

All three businesses rely fundamentally on how effectively the service's seating arrangement can be managed and optimized. As a result, the type of business selected above significantly influences both the structure of the seating layout and the logic of the algorithm used to optimize it, as each context presents different shapes, requirements and customer behaviour.

Cinema:

In cinemas, the seating plans are formed of rows typically arranged in a straight or gently curved layout facing the screen, with most patrons preferring seats toward the middle and back for better viewing angles and comfort. Front rows are often less desirable and single scattered seats in rows may prevent couples or groups from sitting together. An effective seating algorithm should account for these preferences while avoiding scattered single empty seats, which reduce occupancy efficiency.

Airplane:

In airplanes, solo travellers generally prefer window or aisle seats for comfort and accessibility, often avoiding middle seats. Assigning a solo traveller to a middle seat can split a potential pair, making it harder to accommodate couples or groups. This reduces seating efficiency and limits revenue optimization.

Classroom:

In university auditoriums, students often prefer back rows and seats away from crowded areas. While some students prefer to sit in groups, others prefer to leave a blank seat between him/her and others. However, single scattered seats, particularly in middle rows, create issues as some students often attend late and, in order to get seated, he/she has to breach the entire row to get into the middle part, which is where the seats are available. This is a known and generic problem in lecture halls, and since students do not pay for guest talks, booking in advance for a lecture hall space can quickly leave a lot of single scattered seats in the middle of the seating plan.

Regardless of your choice:

As you select one of the seating plans, your algorithm should address the specific scenarios and focus on preventing the scattering of single seats in a way that would optimise the effectiveness of the seating arrangement. However, the accessibility seats, the VIP zones/bigger seats and other constraints demand a customized algorithm for each seating plan rather than overly generic working algorithms. Some seats also get broken randomly. Furthermore, social protocols of our society, such as giving discounts to senior members or having no-children areas on planes, complicate algorithms even more to the extent that AI cannot write this sort of algorithm on its own and your intervention is needed for the optimisation and decision-making process. Remember that generative AI can only solve the problems that exist; it cannot solve approaching, disruptive problems without your intervention

Basic Requirements (Mandatory)

- Use one of the seating plans above for developing your algorithm.
- Make sure to follow the given constraints written on the seating plan, such as how seats are positioned, how many disability seats exist, or adjacent seat requirements.
- Ensure groups of 2–7 people are seated together. Avoid splitting groups unless they exceed the size of a row.
- Solo attendees should not be placed between groups, except in special cases such as VIP or accessible seating.
- An admin override must allow all seating rules to be bypassed if necessary.
- Make sure the thinking process is handled by you and explain this thinking process in a pseudo-code structure or visually by using any tool, such as a block diagram, a flowchart, or even UML Activity.
- Do not use generative AI to blindly construct a solution for the seating algorithms, as they tend to produce ineffective solutions or bypass the requirements, and you will end up submitting a generic solution that matches highly with the similarity of Turnitin, which should not be more than 30%.

Extra Constraints and Requirements (Not mandatory, extra work)

- Support cancellations and rescheduling by demonstrating how your algorithm reallocates released seats effectively.
- Create a full design for the booking algorithm, such as a login, register, movie/flight/guest lecturer interface, where the user can then select their seating. This is extra work, so please only attempt to do it if you finish your algorithm early on. If you do not finish the algorithm first and do this part, it won't be counted as bonus points. The assignment is algorithm, not register/login user interface.

Implementation and Prototype

- You are required to develop a prototype that demonstrates your seating plan logic (not a full software, just the seating book algorithm).
- The prototype you should build should specifically show how the seating algorithm you developed works, but it must reflect the core logic of the seating plan you selected and stress-test the algorithm. Prioritise showing half-full or nearly full circumstances, not only empty ones. **If you do not show half-full or almost-full circumstances regarding how your algorithm behaves, you will not be rewarded with a good mark. The idea here is to**

show how algorithm behaves when single scattered seats block its pathway.

- If the prototype is incomplete or not functional, but your design and rationale are clearly explained, the work may still pass, provided that a video demonstration is done on the basic features of the algorithm.
- A non-functional or missing implementation will limit your final grade to a maximum of a Pass or Merit. To achieve a Distinction, a well-functioning implementation that meets all requirements is essential, in addition to meeting some or all extra requirements given.

Prototype Features to Include (Minimum Viable Product)

- A seating plan as described in the mandatory constraints.
- Selectable seating grid.
- Application of rules, e.g., prevent single-seat gaps.
- Satisfy all mandatory requirements/constraints listed.

Report Structure

Cover Page

- ✓ Student name, ID, programme, assessment title
- ✓ **Video link: VIDEO SHOULD ONLY DEMO THE ALGORITHM. DO NOT SHOW ANYTHING ABOUT YOUR REPORT PLEASE. THE ONLY THING YOU CAN DISCUSS IN THE VIDEO THEORETICALLY SHOULD BE YOUR ALGORITHM LOGIC AND HOW IT IS IMPLEMENTED. DO NOT SPEND HALF OF YOUR VIDEO SHOWING YOUR REPORT. THAT'S A MASSIVE MISTAKE STUDENTS MAKE EACH YEAR.**

Introduction

- Project purpose and selected business context

Requirements

- At least 2 non-functional and 7 functional requirements

System Modelling

- At least 1 Use Case Diagram
- At least 1 Activity Diagram
- At least 3 Sequence Diagrams
- Class Diagrams that show a clear OOP approach, 2 or more

Minimal Viable Product (MVP)

- Technology used
- Seating algorithm design and explanation, You can use pseudo-code or flowchart but you must explain the algorithm logic step by step especially how it behaves in crowded or mass booking situation.
- MVP, a runnable version of the prototype must be explained and displayed (either web, desktop or mobile – whichever you want)
- Describe your solution, challenges faced, and bugs and enhancements, if any.

Testing Strategy

- TDD required
- Optional: BDD or scenario testing

Conclusion and Reflection

- What did this module think you with all the assessment? Was the module leader helpful/supportive? What are you carrying away from this module? Be honest, negativity is not going to affect your mark if it is justly and fairly done. In fact, it could be rewarded because your feedback is valuable to us (E.g. Dr. Cain is a bit weird, I think he has a few screws loose.)

Appendix, full code used for algorithm

Full code of your algorithm must be shared! If you do not share your code, you could be capped. All shared code must be TEXT not SCREENSHOTS!

Please do not put any of the UML diagrams or algorithm logic in the Appendix section. Please submit your report as a PDF file.

Video Submission

❖ **LENGTH: minimum 5 mins and maximum 10 mins. DO NOT EXPLAIN YOUR REPORT IN THE VIDEO! NOT EVEN 1 SECONDS SHOULD BE DEDICATED TO REPORT!**

- Must be uploaded to a public platform, such as YouTube, Vimeo, Panopto, etc.
- Link must be shared on the cover page, not anywhere else in the report.
- Visibility set to “unlisted” or “public” – SHOULD NOT BE PRIVATE.
- Include the video link clearly on the cover page of your PDF submission.
- The video link should be clickable and directly go to the video demonstration.
- Dividing the video into segments is good practice, and a structure is given below.

Video Content

Introduction: I am In this assessment, I Selected

Algorithm Logic Explanation: To develop my algorithm I used Technology. However, before that, I actually developed the logic with pen and paper. Here is my logic ...

Prototype Demonstration: Now that I discussed logic, let’s see how it works... Even on crowded situation... The stress level testing...

Reflection on Module: Dr. Cain is a weird person... I did not like him or his module that much....

Conclusion: In this assessment, I learned...

The prototype demonstration should be on a web or mobile app and recorded in the video. It is strongly suggested to do a full working prototype, but completing a fully working seating plan algorithm is not mandatory to pass this assessment.

Marking Breakdown (Total: 40%)

Component Weight

Design, Documentation & Requirements 20%

Prototype Demonstration 20%